Contrastive Personalized Exercise Recommendation With Reinforcement Learning

Siyu Wu, Jun Wang, and Wei Zhang, *Member, IEEE*

***Abstract*—Personalized exercise recommendation is a challenging task in the field of artificial intelligence in education due to several problems. First, the mainstream approaches focus more on the exercises that students have not mastered, while overlooking their long-term needs during the learning process. Second, it is difficult to capture students' knowledge states caused by sparse interactions with exercises. Moreover, most recommendation methods are dedicated to the performance of the recommendation w.r.t. accuracy, disregarding the students' learning ability. We introduce a new framework called contrastive personalized exercise recommendation with reinforcement learning (RCL4ER) to tackle these issues. Our framework augments the standard recommendation model with an output layer of self-supervised learning and reinforcement learning. The reinforcement allows the supervised layer to focus on specific rewards, acting as a regularizer. The self-supervised layer provides a powerful signal for parameter updating. Three data augmentation methods are used to provide additional data, which are leveraged to conduct contrastive learning and incorporated into reinforcement learning as supplementary information. In addition, we adopt a trained deep knowledge tracing model to capture the changes in students' knowledge states, so as to establish a dynamic reward mechanism. Experiments of our framework based on four recommendation backbones on several public datasets demonstrate the effectiveness of our RCL4ER, which successfully promotes students' capacity and improves the recommendation performance.**

***Index Terms*—Contrastive learning, knowledge tracing, q-learning, recommendation, reinforcement learning.**

I. Introduction

With the development of artificial intelligence (AI), especially the wave of deep learning that began in 2006, Artificial Intelligence in Education (AIED) has become a hot research spot, which holds immense potential for transforming the educational landscape. Personalized exercise recommendation [1], [2], [3] is an important branch in the field of artificial intelligence in education. It aims to recommend appropriate exercises (e.g., problems) for students to answer, which plays an indispensable role in online education platforms like Coursera and intelligent tutoring systems like ASSISTments. As shown in Fig. 1, the exercise recommendation process is seen as a set of interactions between a recommender system (agent) and a student (e.g., stu1). The student answers the exercise recommended by the agent, and then the agent receives feedback from the student and recommends new exercises.

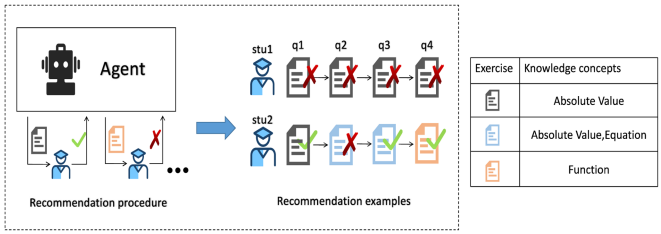

Fig. 1. Example for illustration: A recommendation procedure (left), recommendation examples (center), and knowledge concepts in the exercises (right).

In recent years, there has been a surge in research dedicated to exercise recommendation, encompassing traditional recommendation methods, cognitive diagnosis, knowledge tracing (KT) [4], and reinforcement learning (RL) [5]. Based on the conventions in the traditional recommendation [6], [7], students are regarded as users, test exercises as commodities, and students' scores on test questions as users' scores on commodities. Therefore, traditional recommendation methods [8], which have a simple structure and are easy to understand, can be used to solve the problem of exercise recommendation. Cognitive diagnosis in education is also used in the recommendation of exercises [9], [10]. Based on their assumption, it is believed that students' cognitive levels remain constant over a certain period of time. Therefore, cognitive diagnosis is usually only able to assess the knowledge states of students at a certain moment. In the real environment, however, students' learning is a long-term process, and the knowledge states are constantly changing. Based on this problem, KT technology is proposed [11], [12], [13]. The purpose is to track the change in students' knowledge level through the historical learning data, so as to recommend exercises. Besides, some scholars have also put forward exercise recommendation methods based on RL in the field of education [1], [14], which use layered RL, Q-learning, and other technologies.

Although the above methods have achieved better results in the task of exercise recommendation, there are still some shortcomings.

Manuscript received 16 November 2022; revised 1 May 2023, 3 September 2023, and 4 October 2023; accepted 14 October 2023. Date of publication 23 October 2023; date of current version 5 January 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62072182, Grant 92270119, and Grant 61977058, in part by the Natural Science Foundation of Shanghai under Grant 23ZR1418500, and in part by the Fundamental Research Funds for the Central Universities. *(Corresponding author: Wei Zhang.)*

The authors are with the School of Computer Science and Technology, Shanghai Institute for AI Education, East China Normal University, Shanghai 200062, China (e-mail: 51215901032@stu.ecnu.edu.cn; wongjun@gmail.com; zhangwei.thu2011@gmail.com).

Digital Object Identifier 10.1109/TLT.2023.3326449

1) Most traditional recommendation methods are based on students' preferences, which may result in recommending exercises that are too difficult or too easy [15]. They ignore the information about the exercises and the knowledge concept associated with the student and lack recommendation rationality [16], [17].
2) Generally speaking, methods based on cognitive diagnosis or KT can be roughly divided into two steps: First, the cognitive diagnosis or KT is used to analyze the students' historical information to get the probability of students mastering each knowledge concept. Then, according to the probability of mastery, the exercises involved in the "not mastered" knowledge concept are recommended to the students. While this is reasonable, the single strategy of "recommending unmastered exercises" has certain limitations. For example, in Fig. 1, in the first step, the Agent recommended the exercise containing "absolute value" to stu1. As stu1 was not good at this concept, the algorithm tended to urge him to practice "absolute value" exercises, and even some exercises that he did not complete [18], which was problematic and ignored the student's knowledge throughout the learning process, i.e., long-term learning needs. An ideal RS, therefore, should consider the long-term needs of students. While focusing on recommending unmastered knowledge, it also needs to provide opportunities to explore new knowledge. For example, in Fig. 1, we still want to recommend another exercise with the concept of "equation" to stu2 in the third step. Because he got the exercise with this concept wrong before, but next time he can recommend a new "function" exercise.
3) When students interact with the intelligent tutoring system, they can easily rely on the courses provided by the system, then the number of exercises handled is limited. As a result, this process leads to sparse educational data. For example, the Assist12-13 dataset (details shown in Section V-A) collected from ASSISTments encounters 99.8% sparsity. Representations learned from sparse datasets are often prone to overfitting and have difficulty in accurately capturing students' knowledge states.
4) The existing exercise recommendation methods pay more attention to mimicking student behaviors, but ignore the improvement of students' learning ability [1], [19].

To address the aforementioned issues, this article introduces a novel framework known as contrastive personalized exercise recommendation with RL (RCL4ER).

1) First, for the first and second questions, we formalize the exercise recommendation as a Markov Decision Process (MDP) that interacts between the student and the agent. The model maximizes students' long-term satisfaction with the system by focusing on their long-term needs in the learning process and earning cumulative rewards.
2) Second, for the third problem, the idea of contrastive learning (CL) is introduced. The combination of three data augmentations is used to generate additional data, which are used for CL and also used as additional information for state supplementation of RL, thus alleviating the data sparsity problem to some extent.
3) We set up a corresponding dynamic reward mechanism to break through the fourth problem. The pretrained DKT [20] model is used in the framework to capture students' mastery of the exercises before and after the recommendation. In addition, an evaluation index of learning gain is designed to assess the degree of improvement in students' learning ability by the proposed exercise recommendation method.

In summary, the RCL4ER provides a clearer orientation of students' abilities by considering both their knowledge, the difficulty of the exercises, and their recent problem-solving. The exercises are then recommended to achieve the goal of improving students' learning abilities. This article makes the following key contributions.

1) We propose the RCL4ER framework for solving the exercise recommendation problem. It learns better effective representations by jointly training a self-supervised RL head and a CL head. In this framework, additional information is used to supplement the RL state, while a reward mechanism is designed to more accurately capture changes in the student's knowledge state.
2) Our approach is model-agnostic and can extend existing recommendation models, as demonstrated by the experimental integration of RCL4ER with four representative recommendation models.
3) We conducted experiments on four widely used datasets, focusing on both recommendation quality as well as the improvement of student learning ability. The results show that the performance of the four advanced recommendation models integrating the RCL4ER framework improves on both types of metrics.

The rest of this article is structured as follows. First, the existing research in this domain is presented (see Section II), then the problem definition and preparatory knowledge of the model are introduced (see Section III), and the RCL4ER framework and technical methods are described in detail (see Section IV). Then the article describes the evaluation metrics, datasets, and relevant experimental settings, and verifies the validity of the proposed framework through a large number of experiments (see Sections V and VI). Finally, the work of this article is summarized and the outlook is presented (see Section VII).

II. Relate Work

A. Contrastive Learning

CL is a deep learning technique that operates in a self-supervised and task-independent manner [21]. Its focus is on identifying common features among similar instances and distinguishing differences between nonsimilar instances. CL has shown good performance in natural language processing (NLP) [22], recommender systems [23], computer vision [24], and other fields.

Recently, CL has also been applied to the field of learning technologies, particularly KT. To be specific, the study [25] performed learning history augmentation based on pedagogical

rationales to obtain two correlated representations of the same learning history, which is used in CL. This is beneficial for revealing semantically similar or dissimilar learning histories. Thanks to the integration of the CL target and the KT target, the performance of KT is demonstrated to be improved.

B. Recommender System

Recommender system aims to recommend what users really need. This has resulted in substantial accomplishments across various domains, including POI services [26], e-commerce [27], and social networks [28]. In recent years, recommendation methods based on ranking and rating prediction have been applied to the field of educational technology. These methods have yielded results in test recommendation and adaptive learning [29].

Among the recommendation methods, collaborative filtering is commonly used, which is mainly divided into collaborative filtering based on nearest neighbors and collaborative filtering based on models. The former one is used to achieve exercise recommendation by first calculating the similarity of students on the answer records and then estimating the target students by the resulting scores [30]. The latter one, by constructing a low-dimensional matrix of students and exercises, portrays students' performance in the low-dimensional space, and predicts their scores on the test questions to make exercise recommendations [7]. Jesennia et al. proposed an easy-to-use exercise recommendation system [2]. The main objective is to address the difficulties of computer programming among college students, strengthen their programming skills, and help their cognitive development. The authors divide the study into two phases: student-directed learning and a quantitative study comparing the results with traditional learning methods.

Many neural network-based recommendation methods are also used in the field of education. For example, the ACKRec model based on heterogeneous information networks and graph neural networks [31] is proposed to use graph convolution to learn the representation of entities thus achieving knowledge point recommendation. PLAN-BERT [32] provides a complete mapping of students' future course learning pathways. Due to the essential difference between user goods and student exercises, the use of traditional recommendation algorithms may not take into account other attributes of students, and it is difficult to capture changes in students' knowledge states during the learning process.

C. RL in Education

RL, a subset of machine learning, involves an agent's interaction with an environment to optimize cumulative rewards. In recent years, RL has been widely used in the field of education. In the course recommendation task, in order to make students' test scores improve after completing the courses recommended by the system, CSEAL [33] sets the reward value of actor-critic as the difference between students' test scores before a series of courses and their test scores after learning the courses.

In terms of exercise recommendation, a combination of dynamic attention and hierarchical reinforcement learning (HRL) for DARL [34] is proposed. The model automatically captures student preferences in each interaction between a user and an exercise. The attention weights of corresponding courses at different times are updated adaptively, thus improving the effectiveness and accuracy of course recommendations.

The multiobjective-based RL exercise recommendation, DRER (Deep Reinforcement Learning Framework for Exercise Recommendation with Recurrent Manner) [1], takes the "review and exploration," "difficulty smoothing," and "student engagement" as the objective function. Then, the entire network is trained using this objective function to achieve the optimal combination of multiple recommendation strategies.

The above RL-based approaches have achieved better results, but there may still be effects due to data sparsity. It is also crucial for the construction of the reward mechanism. Compared with previous studies, the proposed RCL4ER framework has the following advantages. First, we use a self-supervised RL approach [35] to select exercises at each step and long-term student demand is obtained indirectly by maximizing the rewards. Second, the RL head makes the supervisory layer's attention focus on the specific rewards. Then a dynamic reward mechanism is designed in the framework, mainly using the pretrained DKT model to obtain the learner's knowledge mastery before and after the recommendation. Thus, the purpose of recommending exercises to learners to enhance their learning ability is achieved. To obtain a better representation, the interaction information obtained by CL is incorporated to mitigate the data sparsity problem to some extent.

III. Preliminaries

For all the specific work performed, this section presents the problem definition and preliminary knowledge of the required methods.

A. Problem Definition

For exercise recommendation, we denote the student set and exercise set as U and Q, respectively. For student $u \in U$, the learning sequence is defined as $u_{1:t} = \{u_1, u_2, \ldots, u_t\}$, where $u_t = (q, r)$ is a learning record and $q \in Q$ represents the student's exercises in timestep t, which are usually associated with knowledge concepts coming from the knowledge concept set K. r_t is the corresponding response, $r_t = 1$ represents the exercise is answered correctly, otherwise, $r_t = 0$.

The exercise recommendation process is shown in Fig. 2. Our agent uses double Q-network, and the environment contains a student pool and an exercise pool. At time step *t*, the agent receives the states of the students, chooses an exercise, and then the students answer, and the agent gets the corresponding reward according to the expression of the students. Specifically, this article designs a dynamic reward mechanism that focuses on improving the students' learning ability. After that, it moves to the next state, and then the agent makes a new recommendation, and so on.

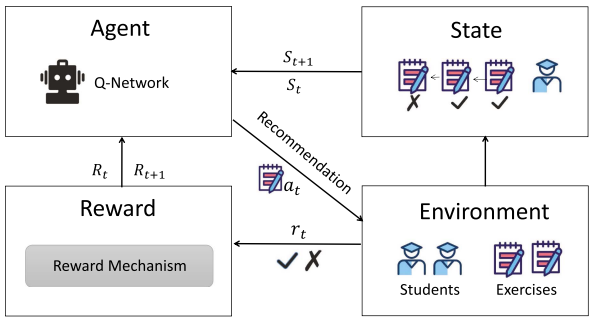

Fig. 2. Overview of the framework for RL.

B. Reinforcement Learning

In this article, the exercise recommendation task is formulated as a MDP [36], where the recommender system is the agent and the students serve as the environments. As shown in Fig. 2, the recommender system interacts with a student in a continuous decision-making process to maximize the discounted cumulative reward. To be precise, the MDP is defined by the tuple consisting of $(S, A, P, R, \rho_0, \gamma)$, where:

1) State S: It indicates the student's learning state space. The student's state, denoted as s_t, is a function of the observed information from time step 1 to t, represented as $s_t = G(u_{1:t})$, and it belongs to the state space "S."
2) Action space A: A is a discrete action space containing all exercises. Based on the state s_t, $a_t \in A$ denotes recommending an exercise q_t to the student. In the offline data, the action at time t can be obtained from the student's interaction with the exercise.
3) Transition probability P: P represents the probability of state transition, i.e., $S \times A \times S \to \mathbb{R}$.
4) Reward R: A reward function, $S \times A \to \mathbb{R}$. The agent recommends exercises (action) to a student and obtains the student's feedback, denoted as the reward $R(s_t, a_t)$. It is crucial for RL and recommender systems to behave well. Therefore, this article designs a novel reward mechanism, which is introduced in Section IV-B.
5) Discount factor γ: γ is the discount factor for reward accumulation, where $\gamma \in [0, 1]$. $\gamma = 0$ means that only timely rewards are considered, and $\gamma = 1$ means that all future rewards are equally considered.
6) Initial state distribution ρ_0: Denoted as ρ_0, represents the probability distribution from which the initial state s_0 is sampled, typically written as $s_0 \sim \rho_0$.

Our goal is to seek an optimal strategy $\pi_\theta(a|s)$ to recommend exercises for students to improve their learning ability. This is realized by maximizing the expected cumulative rewards

$$\max_{\pi_\theta} \mathbb{E}_{\tau \sim \pi_\theta}[\mathbf{R}(\tau)], where\ \mathbf{R}(\tau) = \sum_{t=0}^{|\tau|} \gamma^t \mathbf{r}(\mathrm{s}_t, a_t) \quad (1)$$

where $\theta \in \mathbb{R}^d$ is the parameter of the policy function. The trajectory τ, which consists of state-action pairs $(s_0, a_0, s_1, a_1, \ldots)$, is generated under the influence of the target policy. To elaborate, s_0 is sampled from the initial state distribution ρ_0, a_t is sampled from the policy $\pi_\theta(\cdot|s_t)$, and s_{t+1} is drawn from the distribution $P(\cdot|s_t, a_t)$.

IV. Methodology

Within this section, we provide an in-depth exposition of the RCL4ER framework. It uses double-head self-supervised RL [35] and incorporates data-enhanced data as an additional information. At the same time, CL is combined with RL to learn effective representations. Moreover, the reward mechanism for RL is designed to capture students' mastery of knowledge during the learning process.

This section illustrates the RCL4ER framework in Fig. 3. It first introduces the model architecture (see Section IV-A) and the novel reward mechanism (Section IV-B). Then, the data augmentation module (see Section IV-C) is described in detail. Finally, the learning objectives of RCL4ER (see Section IV-D) are defined. The suggested framework seamlessly integrates with pre-existing recommendation models.

A. Model Architecture

1) Data Augmentation: We employ a data augmentation technique to process a student's learning history $u_{1:t}$ and two related view representations of the same history can be obtained, i.e., u^{+_1} and u^{+_2}, to form a positive pair. It combines three data enhancement strategies: mask, permute, and replace, which reflect different aspects of the learning history and make the framework more robust to the disturbance to improve the corresponding performance. The details of data augmentation are elaborated on later.

2) Embedding Layers: This part contains two embedding layers—the exercise embedding layer and the student-exercise interaction embedding layer. An exercise embedding matrix $E^Q \in \mathbb{R}^{N \times d}$ and a student-exercise interaction matrix $E^I \in \mathbb{R}^{2^{\tilde{}} N \times d}$ (student answer correctness is involved) are constructed to obtain dense representations of students' history sequences, where N denotes the number of exercises and d denotes the embedding dimension. Given an interaction $u_t = (q_t, r_t)$, we can obtain the exercise embedding vector and the interaction embedding vector $e_t^Q = E_{q_t}^Q \in \mathbb{R}^d$ and $e_t^I = E_{q_t + N * r_t}^I \in \mathbb{R}^d$ through the embedding layers.

3) Recommendation Module: Students' learning histories provide insights into various aspects of the knowledge acquisition process, enabling the expression of distinct characteristics among different students. Therefore, the hidden representations of both the exercise and interaction parts are used. A series of exercises and interactions are encoded using an existing recommendation model: The exercise encoder G^Q and the interaction encoder G^I. For a given exercise embedding sequence and interaction embedding sequence, we use the encoder to get the representations of the exercise and interaction as follows:

$$\begin{aligned} h_{1:t}^Q &= G^Q(e_{1:t}^Q) \quad & h_{1:t}^I &= G^I(e_{1:t}^I) \\ \tilde{h}_{1:t}^Q &= G^Q(\tilde{e}_{1:t}^Q) \quad & \tilde{h}_{1:t}^I &= G^I(\tilde{e}_{1:t}^I) \end{aligned} \quad (2)$$

where $\tilde{h}_{1:t}^Q$ and $\tilde{h}_{1:t}^I$ are the hidden state representations of the exercise and interaction after data augmentation.

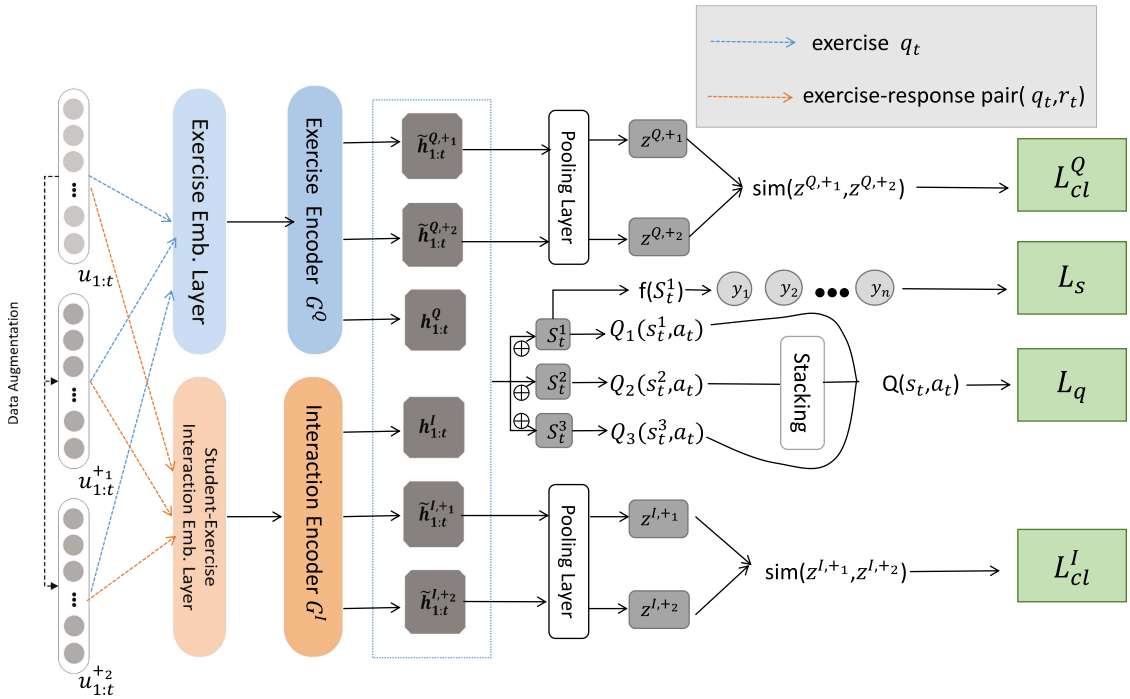

Fig. 3. Overall architecture of RCL4ER. The blue dashed boxes represent the splicing of the respective exercises and interactions. $s^1_t = h^Q_{1:t} \oplus h^I_{1:t}$; $s^2_t = \tilde{h}^{Q,+1}_{1:t} \oplus \tilde{h}^{I,+1}_{1:t}$; $s^3_t = \tilde{h}^{Q,+2}_{1:t} \oplus \tilde{h}^{I,+2}_{1:t}$.

After obtaining the representations, they are concatenated to form the states in RL, i.e., $s^1_t = h^Q_{1:t} \oplus h^I_{1:t}$; $s^2_t = \tilde{h}^{Q,+1}_{1:t} \oplus \tilde{h}^{I,+1}_{1:t}$; $s^3_t = \tilde{h}^{Q,+2}_{1:t} \oplus \tilde{h}^{I,+2}_{1:t}$.

4) The RCL4ER Framework: The exercise recommendation task could be converted into a multiclassification problem, given the representations of student learning histories. The generated classification logits, denoted as $y_{t+1} = [y_1, y_2, \ldots, y_n] \in \mathbb{R}^n$, correspond to the available candidate exercises, with n representing their total count. The classification logits are obtained by $y_{t+1} = f(s_t)$, in which the full connection layer is used. Then, the recommended list for timestamp $t+1$ consists of the top-k exercises selected from y_{t+1}. Finally, we optimize the following cross-entropy loss L_s to train the recommendation model

$$L_s = -\sum_{i=1}^{n} Y_i \log(p_i), \text{where } p_i = \frac{e^{y_i}}{\sum_{i'=1}^{n} e^{y_{i'}}} \tag{3}$$

where $Y_i = 1$ indicates that the student interacts with the ith exercise, and otherwise $Y_i = 0$. The input vector is encoded by the recommendation model G as a potential representation s_t, which can be used as the current state of the RL head. We stack an extra layer to calculate the corresponding Q value

$$Q_z(s_t, a_t) = \delta(s_t W_z + b_z) = \delta(G(e_{1:t})W_z + b_z) \tag{4}$$

where $z \in \{1,2,3\}, s_t \in \{s^1_t, s^2_t, s^3_t\}, e_{1:t} \in \{e^Q_{1:t}, e^I_{1:t}\}$. Both W_z and b_z are learnable parameters and δ is the activation function. Then, the vector is superimposed by Q_1, Q_2, Q_3:

$$Q(s_t, a_t) = [Q_1(s^1_t, a_t), Q_2(s^2_t, a_t), Q_3(s^3_t, a_t)]. \tag{5}$$

It contains the interaction information obtained by the encoder after using the data augmentation method. Subsequently, at each time step t, we extract (s_t, a_t, r_t, s_{t+1}) from the experience buffer D, which is generated using implicit feedback data. Based on this, Q-network computes TD-error from this tuple as the loss of RL:

$$L_q = \left(R_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\right)^2. \tag{6}$$

Sampling training experience (experience replay) improves sample efficiency and training for a fixed target network prevents rapid propagation of approximation errors between two states. The self-supervised head generates recommendations in the form of the top-k exercises. RL serves as a means of regularizer, fine-tuning model G based on the assessment of the recommendation quality of these top-k exercises according to the defined reward mechanism. To improve the stability of learning, we employ the double Q-learning [37] to alternate the training of learnable parameters.

CL is a technique for obtaining meaningful representations by revealing similarities or differences in learning histories. First, the representations denoting the whole exercise and interaction histories are obtained by

$$z^Q = \text{pool}\left(\tilde{h}^Q_{1:t}\right) \qquad z^I = \text{pool}\left(\tilde{h}^I_{1:t}\right) \tag{7}$$

where pool(•) is a mean-polling layer. Then, we formulate the contrastive loss function used in CL. For the positive pair (u^{+1}, u^{+2}), it is instantiated to exercise histories $(z^{Q,+1}, z^{Q,+2})$ and interaction histories $(z^{I,+1}, z^{I,+2})$. The negative representations are the samples of other histories in the same mini-batch

after augmentation: $z^{Q,-} \in Z^{Q,-}$ and $z^{I,-} \in Z^{I,-}$. Upon this, the loss is defined for both the exercise and interaction aspects

$$L_{cl}^{Q} = -log\frac{e^{sim(z^{Q,+_1},z^{Q,+_2})}}{e^{sim(z^{Q,+_1},z^{Q,+_2})}+\sum_{z^{Q,-}\in Z^{Q,-}} e^{sim(z^{Q,+_1},z^{Q,-})}} \tag{8}$$

$$L_{cl}^{I} = -log\frac{e^{sim(z^{I,+_1},z^{I,+_2})}}{e^{sim(z^{I,+_1},z^{I,+_2})}+\sum_{z^{I,-}\in Z^{I,-}} e^{sim(z^{I,+_1},z^{I,-})}} \tag{9}$$

where sim is a cosine similarity function.

B. Reward Mechanism

To make the basic recommendation model G better at recommending exercises to improve students' abilities, we use the trained DKT model [20]. It uses LSTM as a neural network layer to model the interactions of students with the exercises. The reward mechanism is defined as follows:

$$R_t = \frac{\beta}{N}\sum_{i=1}^{N}(p_t^i - p_{t-1}^i)$$
$$p_t = DKT(u_1, u_2, \ldots, u_t) \tag{10}$$

where N is the number of exercises and β is a rescaling factor, which we set to ten empirically. To be specific, first, the sequence of exercises done before time *t* is taken as the input to obtain the students' mastery of each exercise before the recommendation, and the answer information of the recommended exercises can be obtained from it. Then, the interaction sequence at time *t* was input into the DKT to obtain the students' mastery of the exercises after the recommended exercises. Finally, the mean of the difference between all the exercise mastery at time *t* and *t*−1, namely the increment of the current student's knowledge state, is used as the final reward. In addition, we have added additional coefficients.

C. Data Augmentation Module

The choice of data augmentation methods is particularly important. Due to the complexity and characteristics of the exercise recommendation task, the data augmentation methods in CV and NLP are not suitable for the exercise recommendation task. Inspired by Lee et al. [25], our RCL4ER model also uses a combination of multiple data augmentation methods. The proficiency level of each student for the exercises after data augmentation is similar to the previous one and can be added to RL as supplementary information.

1) Exercise Masking: We use the method of random mask used in BERT [38]. For each student learning history sequence, a proportion of the exercises (with a certain ratio of γ_{mask}) are masked out randomly and special markers are placed at the corresponding positions. The newly obtained exercises could be regarded as augmented versions of the original exercises, which are promising for the model to learn more useful information. In addition, this data augmentation also facilitates the estimation of contextual deficits, which can avoid the bias caused by sparse educational data to some extent and learn better representations.

2) Exercise Permutation: Reorder the subsequences in the original history sequence means that the order within the sequence will change somewhat, but the state of student knowledge represented by the interaction sequence will remain similar. For each historical sequence, a randomly disrupted consecutive subsequence $(u_r, u_{r+1}, \ldots, u_{r+L_p-1})$ starting with a length of $L_p = \lfloor \gamma_{per} * t \rfloor$, where γ_{per} is the ratio of disruption. It is also assumed that the student's level of mastery does not change for the answer sequence, as no additional learning material is acquired from elsewhere throughout the recommendation process. In other words, in the learning process, a student has mastered certain knowledge concepts and can solve exercises related to the knowledge concepts, regardless of the order in which the exercises appear. The sequences after using the permute augmentation method yield more information on the representation of the student's learning process.

3) Exercise Replacement: Exercise replacement is done by replacing the original exercise with a simpler or more complex exercise based on the student's original response. The method of exercise replacement allows the inclusion of noise and enhances the model's ability to correct errors, and simultaneously it will improve the model's accuracy and ability to utilize the information. The learning history obtained after replacement still presents a similar knowledge state as the original one. The ratio of replacement is denoted as γ_{rep}.

To implement the replacement approach, only simple cognitive relations are used to construct the knowledge structure. First, for all the exercises in the training dataset, the percentages of being answered correctly and incorrectly (denoting exercise difficulty) are obtained by: $1 - \frac{N(correct)}{N_a}$, where $N(correct)$ denotes the number of the exercise answered correctly and N_a denotes the total number of the exercises answered. According to the difficulty, the exercises are sorted in ascending order, i.e., $\{q(1), q(2), \ldots, q(N)\}$, wherein $q(1)$ is the least difficult one and $q(N)$ is the most difficult.

Exercise replacement means that only the exercise is replaced without changing the student's response. This is because we believe that a student who has mastered a more difficult concept is more likely to answer a relatively easy exercise and vice versa. For example, a student who has mastered the concept of "multiplication" can solve the points related to "addition" more easily ($r_t = 1$). On the contrary, if the student has not mastered the concept of "addition," he is likely to make a mistake when faced with "multiplication"($r_t = 0$). As such, for the student learning sequences, the replacement is chosen randomly with the probability γ_{rep}. Given the cth exercise, if the answer is correct, the original exercise $q(c)$ is replaced with an easier exercise $q(c-k)$, otherwise, it is replaced with a more difficult exercise $q(c+k)$ ($k \in \mathbb{N}^+$).

D. Optimization Objective

The final loss of the RCL4ER framework is the linear sum of L_s, L_q, and L_{cl}:

$$L = L_s + L_q + L_{cl} \tag{11}$$

where $L_{cl} = L_{cl}^{Q} + L_{cl}^{I}$. It is worth noting that after the training process, the recommended exercises are generated by the self-supervised part of the model using both the representation information of the exercises and the interactions. The regularization terms for contrastive and RL are used to avoid the overfitting issue. The RCL4ER framework is adaptable for integration with various existing recommendation models, encompassing session-based and sequence-based models.

V. Experiments

A. Datasets

To assess the efficacy of the proposed framework, we carry out comprehensive experiments using the following four real-world educational datasets.

1) Assist09: Assist09[1] is collected from ASSISTments [39]. ASSISTments is an online tutoring system that provides mathematics education on various types of exercises for grades 1-9. The Assist09 is divided into skill builder data files and nonskill builder data files. The skill builder dataset is derived from the builder exercise set and no further exercises are given after a certain proficiency is met (three exercises answered correctly in a row). After removing the "NAN" knowledge concept and the duplicate sequences due to an exercise containing multiple knowledge concepts, 112 kills are provided and 15 000 exercises are answered by more than 3000 students. The dataset contains a total of 279 658 interactions and the dataset sparsity is 99.5%.

2) Assist12-13: Assist12-13[2] is collected from the same platform as Assist09, but for the 2012–2013 academic year. It contains information such as exercise ID, answer time, multiple skill tags, the number of student attempts, content ID, etc. We process the dataset as having only one knowledge concept for an exercise and further divide the dataset into two independent datasets, i.e., Assist12 and Assist13, according to time information.

3) EdNet_Small: EdNet_small [40] is a large-scale hierarchical dataset collected by Santa (an artificial intelligence tutoring platform) since 2017. It contains 131 417 236 interactions from 784 309 students and stands as the largest publicly available dataset in terms of both total student count and interaction volume. There are four datasets of varying degrees, KT-1, KT-2, KT-3, and KT-4, all of which contain information such as the moment of questioning, student id, exercise id, student-submitted answer, student response time, etc. Because EdNet is too large, we randomly selected 4% of the student data in the EdNet KT-1 dataset to construct the new EdNet_small dataset for our experiments. It contains 2119 students with 277 000 interactions on 142 knowledge points and 11 000 exercises. The sparsity is about 98.8%.

4) BePKT: BePKT [41] contains the programming histories of 906 students from September 2019 to July 2021 collected by an online judge (OJ) system. Student code, exercise text, manually labeled knowledge concepts, and difficulty levels are all included. Different from the previous three datasets, this dataset is used for a real controlled experiment, using programming experts to verify the effectiveness of the proposed framework. To realize this, we select 120 students for testing and a detailed description is given in Section V-I.

TABLE I
Statistics of Datasets

Datasets	Students	Exercises	Concepts	Interactions	Sparsity
Assist09	3879	15505	112	279658	0.995
EdNet_small	2119	11239	142	277540	0.988
Assist12	17886	31065	199	1198453	0.998
Assist13	19113	40769	255	1492306	0.998
BePKT	821	534	88	75845	0.827

For all the above datasets, we remove the interaction sequences with length ≤ 3. Table I illustrates the main characteristics of the datasets. Among the datasets, the first three datasets are commonly adopted in the field of educational data mining. Following previous studies in educational recommendation [42] and other recommendation scenarios [43], we partition each dataset into three distinct subsets: a training set, a validation set, and a test set, which could show some evidence of the effectiveness of the models. In the experiments, we set the ratio of training sets, validation sets, and test sets to 8:1:1.

To validate the proposed framework in a real educational setting, we use the fourth dataset and follow the evaluation protocol [12] to perform a user study, inviting two knowledgeable senior programmers to manually evaluate the recommendation performance between the proposed RCL4ER and a baseline recommendation model. This is somewhat like a controlled experiment. The details of the user study are introduced in Section V-I.

B. Evaluation Metrics

The performance of the proposed framework and other recommendation methods are evaluated from two aspects: Enhancement of student learning ability and recommendation accuracy.

For the former aspect, we use the learning gain of students as the evaluation metric. This is inspired by Luckin et al. [44] who define the learning benefits as $LG = post - pre$, where pre and $post$ refer to students' scores before and after a test. Similarly, we define the learning gain as follows:

$$L\_G = \frac{1}{M}\sum_{i=1}^{M} gain(i) \tag{12}$$

where M is the number of interactions. $gain(\cdot)$ indicates whether students' ability is improved, that is, whether students' exercise mastery is greater after recommendation than before. If so, it is 1; otherwise, it is 0. Among them, the calculation of exercise mastery is the same as the reward computation.

For the latter aspect, the metrics of the hit ratio (HR) and the normalized discounted cumulative gain (NDCG) are used. More precisely, HR@k is utilized to determine if the actual item is positioned within the top k ranks of the recommendation list. NDCG@k takes into account the actual relevance and position

[1][Online]. Available: https://sites.google.com/site/assistmentsdata/home/2009-2010-assistment-data

[2][Online]. Available: https://sites.google.com/site/assistmentsdata/datasets/2012-13-school-data-with-affect

TABLE II
COMPARISON RESULTS ON THE ASSIST09 AND EDNET_SMALL DATASETS

Model	Assisst09							EdNet_small						
	HR@10	NG@10	HR@20	NG@20	HR@50	NG@50	L_G	HR@10	NG@10	HR@20	NG@20	HR@50	NG@50	L_G
GRU	0.18183	0.11512	0.27806	0.13924	0.45345	0.17416	0.502	0.19114	0.14674	0.21254	0.15208	0.25618	0.16161	0.507
RCL4ER(GRU)	**0.19200**	**0.12851**	**0.30003**	**0.14613**	**0.50062**	**0.18568**	**0.535**	**0.20397**	**0.15923**	**0.22152**	**0.16428**	**0.26777**	**0.17320**	**0.539**
SASRec	0.18974	0.12045	0.28590	0.14452	0.45700	0.17920	0.513	0.19213	0.14745	0.20966	0.15196	0.25099	0.15998	0.514
RCL4ER(SASRec)	**0.19844**	**0.13070**	**0.30901**	**0.15143**	**0.51281**	**0.19200**	**0.548**	**0.20427**	**0.15570**	**0.22572**	**0.16107**	**0.27140**	**0.17003**	**0.536**
NItNet	0.18416	0.11751	0.27887	0.14125	0.44865	0.17509	0.509	0.18549	0.14162	0.20099	0.14550	0.23370	0.15192	0.496
RCL4ER(NItNet)	**0.19567**	**0.12482**	**0.30527**	**0.14911**	**0.51787**	**0.19043**	**0.534**	**0.19915**	**0.15511**	**0.21736**	**0.15999**	**0.25992**	**0.16842**	**0.545**
Bert4Rec	0.19271	0.12062	0.29468	0.14664	0.46492	0.18191	0.508	0.19435	0.15161	0.21347	0.15641	0.25889	0.16536	0.500
RCL4ER(Bert4Rec)	**0.19714**	**0.12888**	**0.30841**	**0.15177**	**0.51620**	**0.19212**	**0.528**	**0.20498**	**0.16330**	**0.22596**	**0.16476**	**0.27193**	**0.17379**	**0.537**

NG is an abbreviation for NDCG. Bold fonts indicate the highest scores.

TABLE III
COMPARISON RESULTS ON THE ASSIST12 AND ASSIST13 DATASETS

Model	Assist12							Assist13						
	HR@10	NG@10	HR@20	NG@20	HR@50	NG@50	L_G	HR@10	NG@10	HR@20	NG@20	HR@50	NG@50	L_G
GRU	0.34711	0.28925	0.40801	0.30451	0.52507	0.32765	0.473	0.26145	0.20592	0.32401	0.22160	0.47026	0.24651	0.477
RCL4ER(GRU)	**0.35840**	**0.29725**	**0.42502**	**0.31398**	**0.55622**	**0.33985**	**0.508**	**0.26854**	**0.21054**	**0.33761**	**0.22787**	**0.50301**	**0.25652**	**0.508**
SASRec	0.35294	0.29230	0.42133	0.30947	0.55350	0.33552	0.467	0.26796	0.20863	0.33747	0.22606	0.47691	0.25356	0.421
RCL4ER(SASRec)	**0.36043**	**0.29793**	**0.43034**	**0.31548**	**0.57019**	**0.34305**	**0.511**	**0.27458**	**0.21507**	**0.34640**	**0.23306**	**0.51814**	**0.26296**	**0.469**
NItNet	0.34733	0.287267	0.41375	0.30393	0.53793	0.32844	0.494	0.26670	0.20885	0.33583	0.22619	0.47827	0.25430	0.504
RCL4ER(NItNet)	**0.35904**	**0.29727**	**0.43080**	**0.31528**	**0.56875**	**0.34250**	**0.518**	**0.27297**	**0.21336**	**0.34510**	**0.23143**	**0.51785**	**0.26114**	**0.521**
Bert4Rec	0.34646	0.28734	0.41296	0.30444	0.53725	0.32677	0.489	0.26551	0.20779	0.33456	0.22512	0.47201	0.25223	0.454
RCL4ER(Bert4Rec)	**0.35947**	**0.29720**	**0.43073**	**0.31508**	**0.57294**	**0.34314**	**0.506**	**0.27143**	**0.21125**	**0.34546**	**0.22980**	**0.51720**	**0.26067**	**0.487**

NG is an abbreviation for NDCG. Bold fonts indicate the highest scores.

of each item within the top k positions. It is equivalent to normalizing between different students, with top positions in the recommendation list being assigned with higher scores.

C. Sequential Recommendation Models

We consider four different sequential recommendation models and incorporate each of them into the proposed RCL4ER framework to test its effectiveness:

1) GRU [36]: It is a pioneering model to learn user behavior sequences for sequential recommendation.
2) NItNet [45]: This is a simple and effective model whose network structure is consisted of multiple convolutional layers stacked on each other. Residual connections are also considered to better model sequences.
3) SASRec [46]: This is a competitive sequential recommendation model that builds on the transformer architecture [47], which could capture the long-term dependencies between different interactions in a student behavior sequence.
4) Bert4Rec [48]: It is also a transformer-based sequential recommendation model, but uses bidirectional self-attention to model student behavior.

D. Parameter Settings

For all the datasets, the last ten exercises before the timestamp that needs to be predicted are used as the input sequence. If the sequence is too long, it is truncated, and if it is too short, it is concatenated with padding items. To have a fair comparison, the embedding and hidden layer sizes of all models are set to 64. The size of a mini-batch is set to 256. The learning rate is 0.005 for all the datasets except for Assist12, where the learning rate is 0.01. Adam is used to optimize the model parameters. For the NextItNet model, we use the publicly available source code and keep its parameter settings unchanged. For the GRU model, we set the dimension of its hidden state to 64. In the case of the SASRec model, we utilize one self-attention head. Finally, for the Bert4Rec model, we specify one attention head, two blocks, and a dropout rate of 0.1. For the data augmentation module, we adjust γ_{mask}, γ_{rep}, and γ_{per} within the range of {0.2, 0.3, 0.5}.

E. Overall Performance Comparison

Tables II and III show the overall evaluation results on the four datasets. We observe that the proposed RCL4ER framework can improve the performance of different recommendation models in terms of both learning ability gains and recommendation quality. This is attributed to the fact that:

1) RCL4ER simulates long-term cumulative reward by using a Q-learning head and obtains better learning signals to improve the recommendation performance.
2) A reward mechanism based on the DKT model is tailored for improving students' abilities by giving appropriate reward values.
3) Data augmentation in CL plays a role in providing additional learning histories and supplementing state representations for RL.

In Assist09, the performance improvement was most noticeable, around 3%–13%. In EdNet_small, this framework led to a performance improvement of 6And the Assist12 and Assist13 also maintained a 2% to 6% improvement.

It is observed that on EdNet_small, the results in terms of HR@10 are higher than that of ASSISTment, while the results in terms of HR@20 and HR@50 are significantly lower than that of the other three datasets, and the growth rate is relatively slow. We attribute this phenomenon to the reason that EdNet_small has different data characteristics. To be specific, we find that on EdNet_small, the exercises done by each student exhibit weak correlations and the corresponding knowledge concepts

TABLE IV
RESULTS OF ABLATION STUDY ON THE ASSIST09 AND EDNET_SMALL DATASETS

Models		Assist09							EdNet_small						
		HR@10	NG@10	HR@20	NG@20	HR@50	NG@50	L_G	HR@10	NG@10	HR@20	NG@20	HR@50	NG@50	L_G
GRU	RCL4ER	**0.19200**	**0.12851**	**0.30003**	**0.14613**	**0.50062**	**0.18568**	**0.535**	**0.20397**	**0.15923**	**0.22152**	**0.16428**	**0.26777**	**0.17320**	**0.539**
	- w/o AUG	0.19112	0.11754	0.29653	0.14447	0.49412	0.18332	0.518	0.20018	0.15898	0.22110	0.16349	0.26592	0.17265	0.515
	- w/o CL	0.18944	0.11643	0.29440	0.14274	0.49038	0.18264	0.516	0.19890	0.15714	0.22000	0.16243	0.26538	0.17118	0.513
	- w/o RL	0.18446	0.11751	0.28652	0.14108	0.48660	0.17682	0.513	0.19250	0.14971	0.21564	0.15550	0.26038	0.16433	0.509
SASRec	RCL4ER	**0.19844**	**0.13070**	**0.30901**	**0.15143**	**0.51281**	**0.19200**	**0.548**	**0.20427**	**0.15570**	**0.22572**	**0.16107**	**0.27140**	**0.17003**	**0.536**
	- w/o AUG	0.19411	0.12092	0.29880	0.14713	0.50221	0.18759	0.521	0.19532	0.14794	0.21388	0.15160	0.25367	0.15843	0.521
	- w/o CL	0.19817	0.12376	0.30623	0.15075	0.50848	0.18912	0.519	0.19795	0.15443	0.22108	0.16048	0.26885	0.16986	0.518
	- w/o RL	0.19443	0.12125	0.30197	0.14621	0.50288	0.18662	0.516	0.19708	0.15171	0.21754	0.15685	0.26073	0.16536	0.516
NItNet	RCL4ER	**0.19567**	**0.12482**	**0.30527**	**0.14911**	**0.51787**	**0.19043**	**0.534**	**0.19915**	**0.15511**	**0.21736**	**0.15999**	**0.25992**	**0.16842**	**0.545**
	- w/o AUG	0.19469	0.12216	0.30376	0.14860	0.50777	0.18886	0.518	0.19750	0.15495	0.21694	0.15935	0.25706	0.16725	0.536
	- w/o CL	0.19401	0.12207	0.30283	0.14841	0.50003	0.18762	0.524	0.19708	0.15371	0.21509	0.15845	0.25545	0.16640	0.525
	- w/o RL	0.19205	0.11987	0.29276	0.14511	0.48218	0.18277	0.508	0.19092	0.14838	0.21045	0.15328	0.25158	0.16137	0.512
Bert4Rec	RCL4ER	**0.19714**	**0.12888**	**0.30841**	**0.15177**	**0.51620**	**0.19212**	**0.528**	**0.20498**	**0.16330**	**0.22596**	**0.16476**	**0.27193**	**0.17379**	**0.537**
	- w/o AUG	0.19692	0.12254	0.30671	0.15031	0.50180	0.19104	0.514	0.20429	0.15888	0.22473	0.16253	0.27086	0.17166	0.519
	- w/o CL	0.19619	0.12268	0.30460	0.15010	0.50077	0.19095	0.510	0.20319	0.15550	0.22319	0.16051	0.26984	0.16965	0.516
	- w/o RL	0.19583	0.12051	0.30095	0.14682	0.49186	0.18470	0.510	0.19299	0.14898	0.21463	0.15443	0.25853	0.16308	0.498

Bold fonts indicate the highest scores.

are largely different sometimes. By contrast, on ASSISTment, the exercises within each student's history have shared similar knowledge concepts and the sequences are usually longer.

On the four datasets, we can observe that the Bert4Rec and SASRec models with a transformer architecture have higher recommendation performance than GRU in most cases. This indicates that the transformer and the attention mechanisms can improve the recommendation accuracy to some extent.

In conclusion, the RCL4ER framework showcased in this article consistently outperforms the chosen baseline, resulting in enhanced learning gains and recommendation accuracy. These results underscore the effectiveness and robust generalization of our approach.

F. Ablation Study

This section tests the effectiveness of the main considerations in the proposed RCL4ER framework. We consider the following variants of RCL4ER.

1) "w/o AUG" denotes removing the data augmentation module, including exercise masking, exercise permutation, and exercise replacement.
2) "w/o CL" means removing the contrastive loss from the whole framework.
3) "w/o RL" represents removing the RL part, including the Q-network and TD-error.

We test the three variants on Assist09 and EdNet_small and the results are shown in Table IV. Note that the results on the other two test datasets exhibit similar trends.

Based on the results, we observe that "w/o RL" incurs a larger performance drop in most cases, as compared to the other two variants. This suggests that it is crucial to use RL and introduce RL head to simulate long-term cumulative rewards, which can help learn additional useful information and improve recommendation performance. Moreover, the removal of CL also causes some performance degradation. In particular, on EdNet_small, a larger decrease is observed w.r.t. L_G. These results reveal that CL facilitates the learning of students' knowledge representation information. Besides, data augmentation also exhibits a positive contribution to the final performance. Note that although the performance is reduced by removing one part of the framework, it is still better than the baseline methods without using the framework.

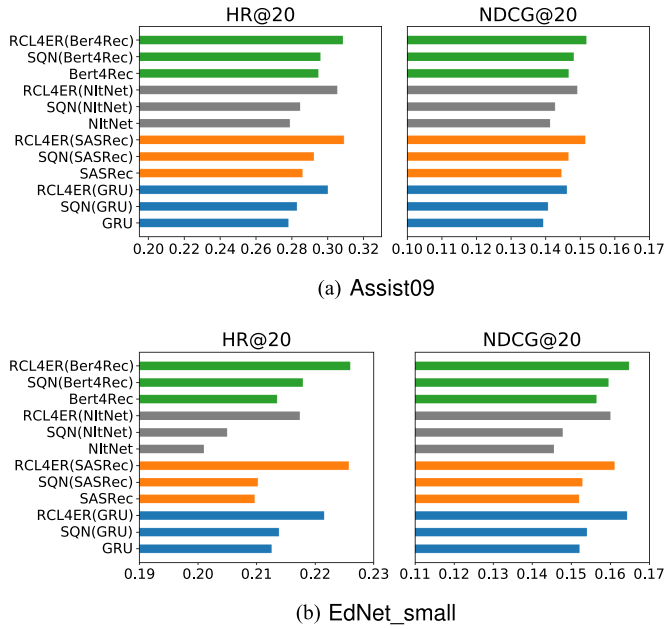

Fig. 4. Performance comparison between "SQN" and the proposed framework on the Assist09 and EdNet_small datasets.

In addition, we test the effectiveness of interaction embedding with CL by removing both of them (namely "SQN"), which could be regarded as a self-supervised RL method [35], but using the proposed reward mechanism. Fig. 4 shows the results of the adopted four sequential recommendation models using "SQN" and the proposed framework, which are tested on the Assist09 and EdNet_small datasets in terms of the HR@20 and NG@20 metrics. We can observe that the basic recommendation models are inferior to "SQN." This means that the introduction of the RL head successfully focuses on the long-term needs of students and gets learning signals, making it work better. Moreover, our framework still outperforms "SQN" significantly, indicating that the combination of RL and CL and the addition of interaction embedding are more conducive to the

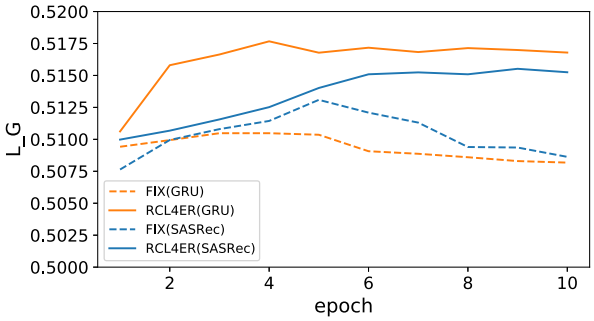

Fig. 5. Results of learning gain during the learning process on EdNet_small.

performance improvement of the sequential recommendation models.

G. Impact of Reward Mechanism

We carry out experiments to the efficacy of the designed reward mechanism for improving students' learning gains. To realize this, we consider an alternative and straightforward reward design, which uses a fixed reward mechanism. Specifically, if the answer to the exercise is judged to be correct, reward=1, otherwise reward=0. We term the method using the fixed reward as "FIX." The two sequential recommendation models—GRU and SASRec—are selected for testing the performance of the reward designs.

Fig. 5 shows the results of "FIX" and the proposed reward mechanism in terms of the evaluation metric L_G. As can be seen, as the training process goes on, RCL4ER(GRU) and RCL4ER(SASRec) outperform FIX(GRU) and FIX(SASRec) with notable margins, respectively. This demonstrates that considering the change in knowledge mastery during the student learning process is critical for achieving better learning gain. Moreover, the results of RCL4ER(GRU) and RCL4ER(SASRec) exhibit an overall upward trend in the first several epochs and then become stable. Contrary to the proposed reward mechanism, "FIX" even decreases the learning gain after training for several epochs.

H. Analysis of Data Augmentation Strategies

This section shows the performance of each data augmentation strategy used in this article and how their ratios (γ_{mask}, γ_{per}, and γ_{rep}) affect the performance. We use "mask," "permute," and "replace" to denote exercise masking strategy, exercise permutation, and exercise replacement, respectively. GRU is used as the basic recommendation model and HR@10 is used as the evaluation metric. The test is conducted on Assist09 and EdNet_small, where we vary the ratios in the range of 0.1 to 0.9.

As the results are shown in Fig. 6, the methods using only one data augmentation strategy are inferior to the overall method. This phenomenon indicates the advantage of combining multiple data augmentation strategies. Moreover, the performance of each data augmentation strategy varies across different datasets and the influence of ratios is also different. This might suggest that each augmentation method focuses on different characteristics of student learning sequences.

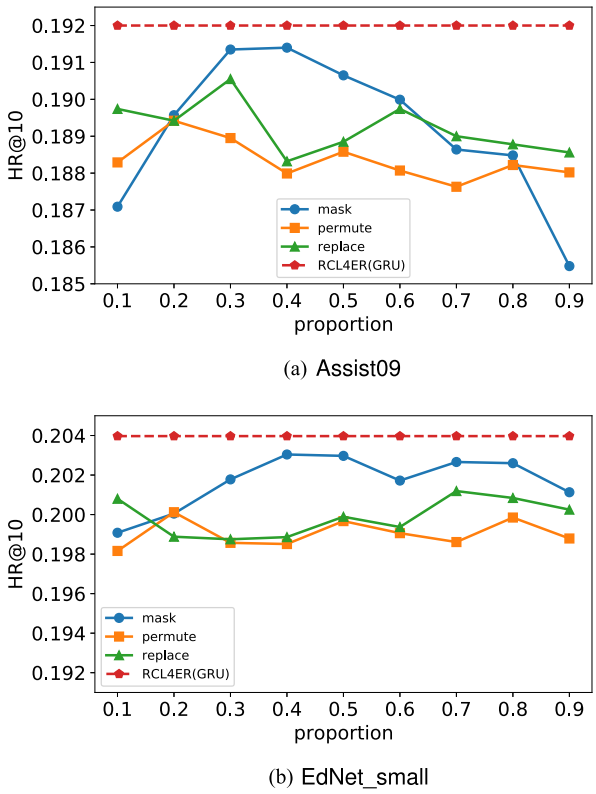

Fig. 6. Impact of the different augmentation methods with different proportions in terms of HR@10.

I. User Study

Despite the above experiments conducted on three publicly available datasets, we additionally validate the proposed framework in a real educational setting by a user study. In the user study, we select 120 students who have programming histories in an online judge (OJ) system [41]. Based on their programming histories, we choose RCL4ER(GRU) to generate exercise recommendation and use the recommended exercises from the baseline recommendation model GRU for comparisons. This is similar to a controlled experiment, where RCL4ER(GRU) corresponds to the experimental group and GRU corresponds to the reference group. It is not easy for the students themselves to judge the quality of the exercise recommendations due to the following reason. Contrary to interest-driven recommendation like product recommendation, exercise recommendation aims at not only satisfying student interests but also improving their knowledge mastery. As such, we invite two knowledgeable senior programmers who have won medals in the ACM-ICPC Asia Regional Contest. It is worth mentioning that we do not use the ASSISTment and EdNet datasets because they do not have enough exercise content information for judgment.

TABLE V
OVERALL USER EVALUATION OF RECOMMENDED EXERCISES

T_1	T_2		Total
	R1_better	R2_better	
R1_better	30	11	41
R2_better	13	66	79
Total	43	77	120

TABLE VI
USER EVALUATION OF RECOMMENDED EXERCISES ON EACH ASPECT

Aspect	T_1		T_2	
	R_1	R_2	R_1	R_2
Str_weak	2.683	3.242	2.658	3.016
Re_his	2.825	3.316	2.850	3.125
Exer_diff	3.000	3.192	3.041	3.183
Abi_impro	2.883	3.233	2.758	3.042

For the exercise recommendation of each student, the two programmers evaluate the effectiveness from the following four aspects:

1) "Str_weak" (strengthen weak knowledge concept): It denotes how the recommended exercises include knowledge concepts that students have not mastered or have answered incorrectly.
2) "Re_his" (recall of historical knowledge concept): It denotes how the recommended exercises can take into account the forgetting phenomenon of students and help them to recall what they have learned.
3) "Exer_diff" (difficulty of recommended exercises): It represents how the difficulty level of the recommended exercises matches the knowledge mastery level of a student.
4) "Abi_impro" (improve students' learning ability): It represents how the overall exercise recommendation can facilitate student ability improvement.

The rating scores for each aspect range from 1 to 5 (the larger the score, the higher the evaluation). Moreover, the two programmers are required to provide an overall evaluation of the recommended exercises for each student for the two methods, i.e., RCL4ER(GRU) and GRU.

The results of the user evaluation are shown in Tables V and VI. We use "T_1" to denote the first programmer and "T_2" to denote the second programmer. "R1" and "R2" correspond to the GRU and RCL4ER(GRU) recommendation methods, respectively. Moreover, "R1_better" indicates that the recommendation result of GRU is thought to be better than that of RCL4ER(GRU), while "R2_better" is the opposite. Before analyzing the comparison of the two recommendation methods, we first verify the reliability of the user evaluation. This is realized by calculating the annotation agreement.

1) Annotation Agreement: To test the label annotation agreement of the two programmers, we adopt the commonly used Fleiss' Kappa measure, which is defined as $k = 1 - \frac{1-p_0}{1-p_e} = \frac{p_0-p_e}{1-p_e}$, where p_0 represents the observed consistency among annotators, and p_e is the consistency based on assumptions and caused by random factors. Based on the results shown in Table V, the calculated value of Fleiss' Kappa is 0.561, indicating that the two programmers reach an agreement to a certain extent and the label annotation results are reliable.

2) User Evaluation: First, as shown in Table V, both the two knowledgeable programmers agree that RCL4ER(GRU) provide better exercise recommendation than GRU, since the number of students corresponding to "R2_better" is significantly larger than that of "R1_better." Second, as can be seen in Table VI, RCL4ER(GRU) ("R_2") obtains consistently higher rating scores than GRU ("R_1") on all four aspects. These results reflect that both programmers think the exercises recommended by RCL4ER(GRU) could better facilitate improving students' knowledge mastery and learning abilities.

VI. APPLICATION

To illustrate the practical significance of the proposed personalized exercise recommendation method, we describe how it could be used in a real educational system, i.e., the OJ system of East China Normal University,[3] which is depicted in Fig. 7 and regarded as our future work. The main application is personalized modeling of students to recommend appropriate programming exercises for the purpose of tailoring instructions to their needs. The proposed method could leverage the data collected from the OJ platform to train the recommendation model. The data mainly includes the behaviors (e.g., exercise practice) of all the students involved in the platform. After training, the model can accept the historical records of a student and output a recommendation list containing top k exercises that are suitable for the student. The recommendation model can adjust the recommended exercises according to the latest programming behaviors of students, and the model training process could be performed only on a daily/weekly basis. The detailed steps about how to utilize the exercise recommendation method could be illustrated as follows.

Step 1: A student selects exercises in the OJ system to answer and submits the written code.
Step 2: The system gets the student's historical exercise records, including the exercise ID, the knowledge concepts, the correctness of answers, the submitted code, etc.
Step 3: Based on the learning history, RCL4ER calculates the current learning state of the student.
Step 4: According to the learning state, RCL4ER generates the top-k recommended exercises, which will be displayed on the recommendation page of the system. An example is shown in Fig. 7.
Step 5: The student could choose to click some recommended exercises for further practice and submit new code solutions.
Step 6: The system then automatically judges the answer correctness of the submitted code and stores the new exercise record, and then RCL4ER makes new exercise recommendation based on the updated student learning history.

As such, the system could carry out dynamic interactions between students and the OJ system. The length of the historical exercise sequence and the number of recommended exercises

[3][Online]. Available: https://acm.ecnu.edu.cn/

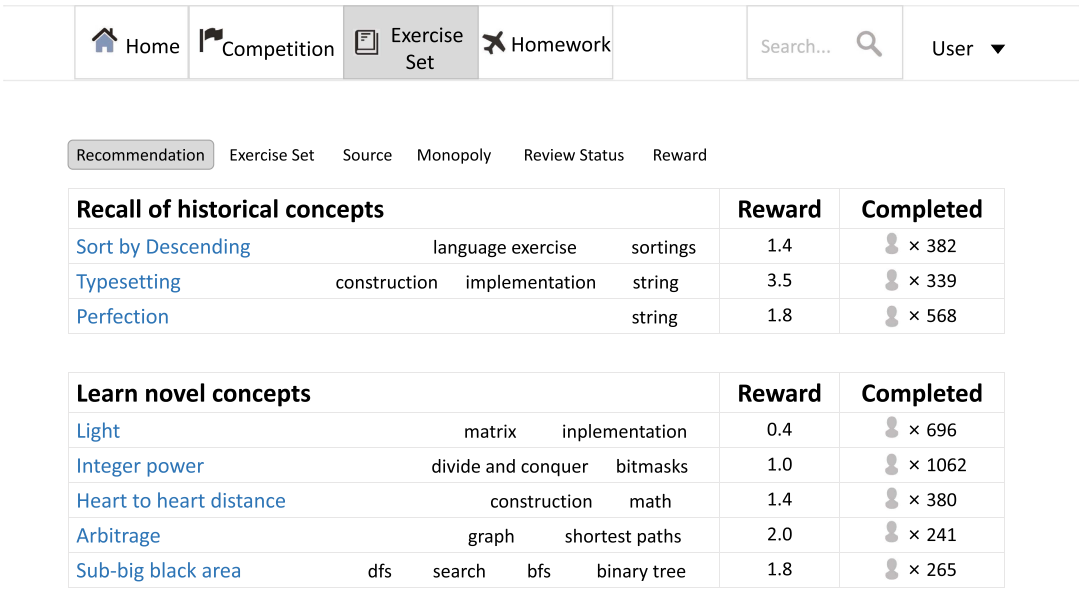

Fig. 7. Example of applying the proposed exercise recommendation method.

can be set freely by the teachers teaching programming courses. Moreover, the teachers could take the recommended exercises as references to determine which programming exercises are suitable for a specific student. This can reduce the teaching burden of teachers and allow them to have more time and energy to focus on students' learning progress, as well as provide better guidance and support for students. In total, a promising manner to achieve personalized education and adaptive learning.

VII. Conclusion

In conclusion, we propose the RCL4ER framework for personalized exercise recommendation. Our framework combines RL and CL techniques to enhance the existing recommendation model and achieve high-quality representations. We introduce desirable attributes into the model through the RL head as a regularizer, and employ three data augmentation methods to generate additional data, which are leveraged to conduct contrastive learning while incorporated into RL as complementary information. Additionally, we use a novel reward mechanism based on the DKT model to guide the model learning process. We evaluate the effectiveness of the RCL4ER framework by seamlessly integrating it with four state-of-the-art recommendation models. Through comprehensive experiments on four publicly available educational datasets, we validate its performance. Furthermore, we test the proposed model in real-world conditions and achieve improved recommendation performance and learning gain.

In future research, we aim to integrate knowledge from the field of cognitive psychology and introduce more learner-related information into the framework. Specifically, we will consider students' long-term memory and personality in actual learning to make the recommendation effect more reliable.

References

[1] Z. Huang et al., "Exploring multi-objective exercise recommendations in online education systems," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 1261–1270.

[2] J. Cárdenas-Cobo, A. Puris, P. Novoa-Hernández, J. A. Galindo, and D. Benavides, "Recommender systems and scratch: An integrated approach for enhancing computer programming learning," *IEEE Trans. Learn. Technol.*, vol. 13, no. 2, pp. 387–403, Apr.–Jun. 2020.

[3] H. Wan, Z. Zhong, L. Tang, and X. Gao, "Pedagogical interventions in SPOCs: Learning behavior dashboards and knowledge tracing support exercise recommendation," *IEEE Trans. Learn. Technol.*, vol. 16, no. 3, pp. 431–442, Jun. 2023.

[4] G. Abdelrahman, Q. Wang, and B. P. Nunes, "Knowledge tracing: A survey," *ACM Comput. Surv.*, vol. 55, no. 11, pp. 224:1–224:37, 2023.

[5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT press, 2018.

[6] M. Vukicevic, M. Jovanovic, B. Delibasic, and M. Suknovic, "Recommender system for selection of the right study program for higher education students," in *RapidMiner: Data Mining Use Cases and Business Analytics Applications*, London, UK: Chapman & Hall, 2013, pp. 145–156.

[7] A. Walker, M. M. Recker, K. Lawless, and D. Wiley, "Collaborative information filtering: A review and an educational application," *Int. J. Artif. Intell. Educ.*, vol. 14, no. 1, pp. 3–28, 2004.

[8] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "Grouplens: An open architecture for collaborative filtering of netnews," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 1994, pp. 175–186.

[9] T.-Y. Zhu et al., "Congnitive diagnosis based personalized question recommendation," *Comput. Sci.*, vol. 41, no. 1, pp. 176–191, 2017.

[10] W. Kang, L. Zhang, B. Li, J. Chen, X. Sun, and J. Feng, "Personalized exercise recommendation via implicit skills," in *Proc. ACM Turing Celebration Conf.-China*, 2019, pp. 1–6.

[11] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Model. User-Adapted Interact.*, vol. 4, pp. 253–278, 1994.

[12] Y. Lu, P. Chen, Y. Pian, and V. W. Zheng, "CMKT: Concept map driven knowledge tracing," *IEEE Trans. Learn. Technol.*, vol. 15, no. 4, pp. 467–480, Aug. 2022.

[13] J. Cui, Z. Chen, A. Zhou, J. Wang, and W. Zhang, "Fine-grained interaction modeling with multi-relational transformer for knowledge tracing," *ACM Trans. Inf. Syst.*, vol. 41, no. 4, pp. 1–26, 2023.

[14] J. Zhang, B. Hao, B. Chen, C. Li, H. Chen, and J. Sun, "Hierarchical reinforcement learning for course recommendation in MOOCs," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 01, 2019, pp. 435–442.

[15] W. Zheng, Q. Du, Y. Fan, L. Tan, C. Xia, and F. Yang, "A personalized programming exercise recommendation algorithm based on knowledge structure tree," *J. Intell. Fuzzy Syst.*, vol. 42, no. 3, pp. 2169–2180, 2022.

[16] N. Thai-Nghe, L. Drumond, T. Horváth, A. Krohn-Grimberghe, A. Nanopoulos, and L. Schmidt-Thieme, "Factorization techniques for predicting student performance," in *Educational Recommender Systems and Technologies: Practices and Challenges*. Hershey, PA, USA: IGI Global, 2012, pp. 129–153.

[17] N. Thai-Nghe, L. Drumond, A. Krohn-Grimberghe, and L. Schmidt-Thieme, "Recommender system for predicting student performance," *Procedia Comput. Sci.*, vol. 1, no. 2, pp. 2811–2819, 2010.
[18] Y. Chen, X. Li, J. Liu, and Z. Ying, "Recommendation system for adaptive learning," *Appl. Psychol. Meas.*, vol. 42, no. 1, pp. 24–41, 2018.
[19] S. Mao, P. Hao, and C. Bai, "Deep correlation based concept recommendation for MOOCs," in *Proc. Int. Conf. Softw. Eng. Knowl. Eng.*, 2022, pp. 622–627.
[20] D. Agarwal, R. S. Baker, and A. Muraleedharan, "Dynamic knowledge tracing through data driven recency weights," presented at the Int. Conf. Educ. Data Mining, 2020, pp. 725–729.
[21] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.
[22] T. Gao, X. Yao, and D. Chen, "Simcse: Simple contrastive learning of sentence embeddings," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 6894–6910.
[23] X. Xie et al., "Contrastive learning for sequential recommendation," in *Proc. IEEE Int. Conf. Data Eng.*, 2022, pp. 1259–1273.
[24] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.
[25] W. Lee, J. Chun, Y. Lee, K. Park, and S. Park, "Contrastive learning for knowledge tracing," in *Proc. ACM Web Conf.*, 2022, pp. 2330–2338.
[26] M. Hang, I. Pytlarz, and J. Neville, "Exploring student check-in behavior for improved point-of-interest prediction," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 321–330.
[27] X. Zhao, L. Zhang, Z. Ding, L. Xia, J. Tang, and D. Yin, "Recommendations with negative feedback via pairwise deep reinforcement learning," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 1040–1048.
[28] L. Wu, L. Chen, R. Hong, Y. Fu, X. Xie, and M. Wang, "A hierarchical attention model for social contextual image recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 10, pp. 1854–1867, Oct. 2019.
[29] F. Okubo, T. Shiino, T. Minematsu, Y. Taniguchi, and A. Shimada, "Adaptive learning support system based on automatic recommendation of personalized review materials," *IEEE Trans. Learn. Technol.*, vol. 16, no. 1, pp. 92–105, Feb. 2023.
[30] D. Wu, J. Lu, and G. Zhang, "A fuzzy tree matching-based personalized e-learning recommender system," *IEEE Trans. Fuzzy Syst.*, vol. 23, no. 6, pp. 2412–2426, Dec. 2015.
[31] Y. Zhou, C. Huang, Q. Hu, J. Zhu, and Y. Tang, "Personalized learning full-path recommendation model based on lstm neural networks," *Inf. Sci.*, vol. 444, pp. 135–152, 2018.
[32] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, 2013.
[33] Q. Liu et al., "Exploiting cognitive structure for adaptive learning," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 627–635.
[34] Y. Lin, S. Feng, F. Lin, W. Zeng, Y. Liu, and P. Wu, "Adaptive course recommendation in MOOCs," *Knowl.-Based Syst.*, vol. 224, 2021, Art. no. 107085.
[35] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Self-supervised reinforcement learning for recommender systems," in *Proc. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2020, pp. 931–940.
[36] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based recommendations with recurrent neural networks," in *Proc. Int. Conf. Learn. Representations*, 2016.
[37] H. Hasselt, "Double Q-learning," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, 2010, pp. 2613–2621.
[38] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
[39] M. Feng, N. Heffernan, and K. Koedinger, "Addressing the assessment challenge with an online system that tutors as it assesses," *User Model. User-Adapted Interact.*, vol. 19, pp. 243–266, 2009.
[40] Y. Choi et al., "EdNet: A large-scale hierarchical dataset in education," in *Proc. Int. Conf. Artif. Intell. Educ.*, 2020, pp. 69–73.
[41] R. Zhu et al., "Programming knowledge tracing: A comprehensive dataset and a new model," in *Proc. IEEE Int. Conf. Data Mining Workshops*, 2022, pp. 298–307.
[42] F. Abel, I. I. Bittencourt, E. d. B. Costa, N. Henze, D. Krause, and J. Vassileva, "Recommendations in online discussion forums for e-learning systems," *IEEE Trans. Learn. Technol.*, vol. 3, no. 2, pp. 165–176, Apr.–Jun. 2010.
[43] G. Shani and A. Gunawardana, "Evaluating recommendation systems," in *Recommender Systems Handbook*. New York, NY, USA: Springer, 2011, pp. 257–297.
[44] R. Luckin, "Beyond the code-and-count analysis of tutoring dialogues," in *Artificial Intelligence in Education: Building Technology Rich Learning Contexts That Work*, Amsterdam, The Netherlands: IOS Press, 2007, pp. 349–356.
[45] F. Yuan, A. Karatzoglou, I. Arapakis, J. M. Jose, and X. He, "A simple convolutional generative network for next item recommendation," in *Proc. ACM Int. Conf. Web Search Data Mining*, 2019, pp. 582–590.
[46] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proc. IEEE Int. Conf. Data Mining*, 2018, pp. 197–206.
[47] A. Vaswani et al., "Attention is all you need," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
[48] F. Sun et al., "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 1441–1450.

Siyu Wu received the bachelor's degree in software engineering from Chongqing Normal University, Chongqing, China, in 2017. She is currently working toward the master's degree in computer science and technology with the School of Computer Science and Technology, East China Normal University, Shanghai, China.

Her research interests mainly include recommendation systems and artificial intelligence for education.

Jun Wang received the Ph.D. degree in electrical engineering from Columbia University, New York, NY, USA, in 2011.

He is currently a Professor with the School of Computer Science and Technology, East China Normal University and an Adjunct Faculty Member of Columbia University. From 2010 to 2014, he was a Research Staff Member at IBM T. J. Watson Research Center, Yorktown Heights, NY, USA. His research interests include machine learning, data mining, mobile intelligence, and computer vision.

Wei Zhang (Member, IEEE) received the Ph.D. degree in computer science and technology from Tsinghua University, Beijing, China, in 2016.

He is currently a Professor with the School of Computer Science and Technology, East China Normal University, Shanghai, China. His research interests mainly include user data mining and machine learning applications.

Dr. Zhang is a senior member of China Computer Federation.